

SlowLoris Active Response

From Detection to Automated Mitigation Using Suricata, Wazuh, and n8n

Shallum Safdar

Overview

This article walks through a complete security pipeline built from scratch: a custom Suricata IDS signature that detects SlowLoris HTTP flood attacks, a Wazuh SIEM agent that ships those alerts to a central manager, and an n8n SOAR workflow that automatically blocks the attacker's IP via SSH and sends a structured email notification, all within seconds of a real attack being detected.

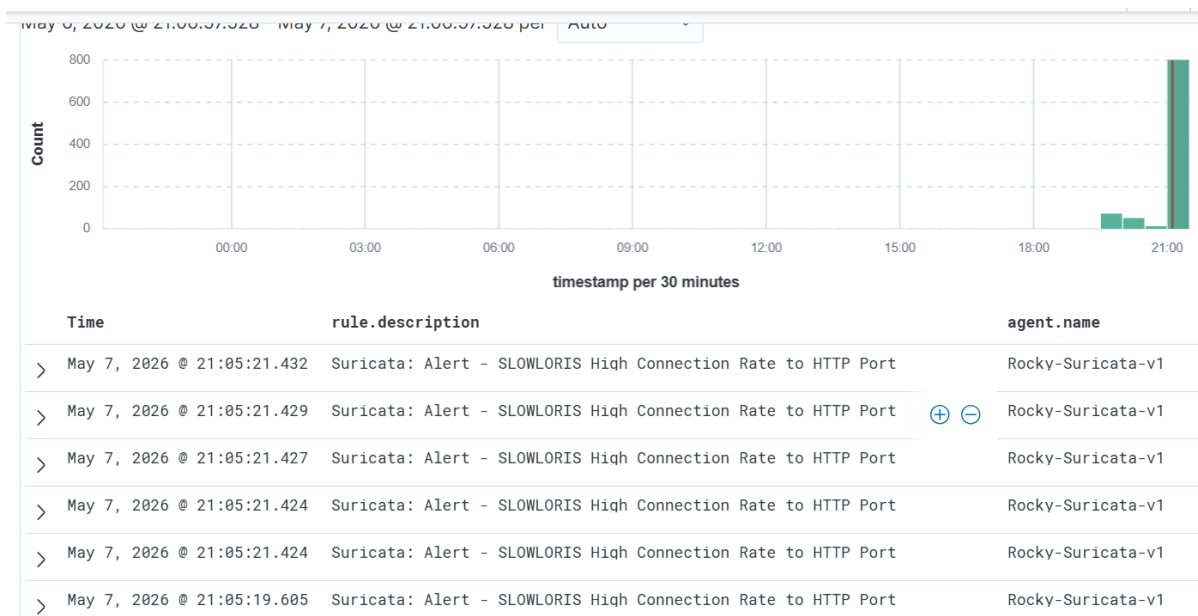
The goal was to build something end-to-end: not just detection, not just alerting, but a full automated response loop that requires zero human intervention once an attack is live.

Architecture and Lab Setup

The lab uses four machines on the same subnet (150.1.7.0/24):

- **Rocky Linux (150.1.7.140):** Rocky Linux (150.1.7.140): runs Suricata 8 in promiscuous mode on ens192, with a Wazuh agent shipping Suricata's eve.json log to the Wazuh manager.
- **Wazuh Server (150.1.7.99):** Wazuh Server (150.1.7.99): receives and indexes Suricata alerts into Elasticsearch, queryable via the Wazuh API.
- **Metasploitable 2 (150.1.7.104):** Metasploitable 2 (150.1.7.104): the victim server, running an HTTP service on port 80 that SlowLoris targets.
- **Kali Linux (150.1.7.101):** Kali Linux (150.1.7.101): the attacker machine, running pentmenu to launch SlowLoris.

n8n runs separately and polls the Wazuh Elasticsearch API every 15 seconds. When it finds a fresh SlowLoris alert, it executes an SSH command on Metasploitable to add an iptables DROP rule for the attacker's IP, then sends an email with full alert details.



Full pipeline: Kali attacker, Suricata IDS, Wazuh SIEM, n8n automation, Metasploitable victim

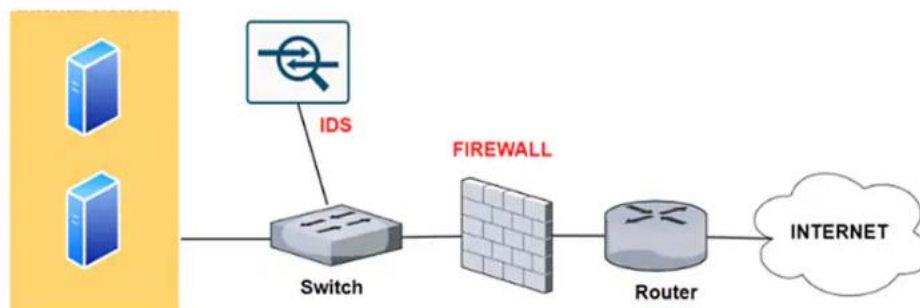
Understanding Suricata Rule Syntax

Before writing a custom signature, it helps to understand the structure of a Suricata rule. The syntax mirrors Snort, which uses PCRE at its core. Every rule has two parts: a header and an options body.

Snort Rule Syntax

Snort follows a specific syntax. In away, it's similar to firewall rules, but the Options body provides more power than a firewall. The syntax is shown below.

Header							Options
Rule Action	Protocol	Source IP Address	Source Port	Flow (Direction)	Destination IP Address	Destination Port	Additional Tests, Output Messages, Etc.



Snort/Suricata rule syntax: header (action, protocol, IPs, ports, direction) and options body

The header defines what traffic to match: the action (alert, drop, pass), the protocol (tcp, udp, icmp), source and destination IP/port, and the direction of flow. The options body is where the real detection logic lives: message strings, payload matching, thresholds, flow state, and the signature ID.

Installing Suricata 8 on Rocky Linux

Suricata 8 isn't in the default Rocky Linux repos, so it needs the OISF COPR repository. The steps below install Suricata, download the Emerging Threats open ruleset, and configure it to listen on the correct network interface.

Install and configure

```
sudo dnf install 'dnf-command(copr)'  
sudo dnf copr enable @oisf/suricata-8.0  
sudo dnf install epel-release  
sudo dnf install suricata  
mkdir -p /etc/suricata/rules/
```

Download the Emerging Threats rules for Suricata 8:

```
cd /tmp && curl -LO https://rules.emergingthreats.net/open/suricata-  
8.0.4/emerging.rules.tar.gz  
sudo tar -xvzf emerging.rules.tar.gz && sudo mv rules/*.rules /etc/suricata/rules/  
chown -R suricata:suricata /etc/suricata/rules/*  
sudo chmod 640 /etc/suricata/rules/*.rules
```

Configure suricata.yaml

Edit `/etc/suricata/suricata.yaml` and set the correct network range, rule path, and capture interface:

```
HOME_NET: "[150.1.7.0/24]"  
EXTERNAL_NET: "any"  
default-rule-path: /etc/suricata/rules  
rule-files:      - "*.rules"  
af-packet:      - interface: ens192
```

Also edit `/etc/sysconfig/suricata` and set:

```
OPTIONS="-i ens192 --user suricata"
```

```
sudo systemctl daemon-reload  
sudo systemctl restart suricata  
sudo systemctl status suricata
```

```
shal@localhost:~ — sudo systemctl status suricata
[sudo] password for shal:
[shal@localhost ~]$ sudo cat /etc/suricata/rules/local.rules
# Rule: POD {Ping of Death}
alert icmp any any -> any any (msg:"Ping of Death - Oversized ICMP"; dsize:>1000
; itype:8; sid:1000002; rev:1;)
[shal@localhost ~]$ sudo systemctl daemon-reload
[shal@localhost ~]$ sudo systemctl restart suricata
[shal@localhost ~]$ sudo systemctl status suricata
• suricata.service - Suricata Intrusion Detection Service
  Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset>
  Active: active (running) since Thu 2026-05-07 20:13:21 PKT; 4s ago
    Docs: man:suricata(1)
  Process: 4777 ExecStartPre=/bin/rm -f /var/run/suricata.pid (code=exited, s>
 Main PID: 4778 (Suricata-Main)
    Tasks: 1 (limit: 22468)
   Memory: 246.4M (peak: 246.4M)
```

```
1 # Rule: POD {Ping of Death}
2 alert icmp any any -> any any (msg:"Ping of Death - Oversized ICMP"; dsize:>1000; itype:
  8; sid:1000002; rev:1;)
3 # Create local rules file
4 sudo nano /etc/suricata/rules/local.rules
5
6 # SlowLoris - fixed
7 alert tcp any any -> $HOME_NET 80 (msg:"SLOWLORIS High Connection Rate to HTTP Port"; \
8   flow:to_server; \
9   flags:S; \
10  threshold:type threshold, track by_src, count 20, seconds 10; \
11  sid:1000011; rev:2;)
12 |
```

Suricata running and active, with local.rules file showing both the Ping of Death and SlowLoris signatures

Connecting the Rocky Linux Agent to Wazuh

With Suricata running, the Rocky Linux machine needs a Wazuh agent installed so it can forward Suricata alerts to the Wazuh manager.

```
curl -o wazuh-agent-4.14.4-1.x86_64.rpm \ https://packages.wazuh.com/4.x/yum/wazuh-agent-
4.14.4-1.x86_64.rpm \ && sudo WAZUH_MANAGER='150.1.7.99' \
WAZUH_AGENT_NAME='RockyLinux-Suricata2-9.3' \ rpm -ihv wazuh-agent-4.14.4-1.x86_64.rpm
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Once the agent appears in the Wazuh manager dashboard, configure it to read Suricata's eve.json log by editing `/var/ossec/etc/ossec.conf`:

```
<ossec_config> ... <localfile> <log_format>json</log_format>
<location>/var/log/suricata/eve.json</location> </localfile> ... </ossec_config>
sudo systemctl restart wazuh-agent
```

Writing Custom Suricata Signatures

Ping of Death (for initial testing)

Before tackling SlowLoris, a simpler Ping of Death rule was created first to verify the full pipeline: Suricata detects it, Wazuh receives it, and alerts appear in the dashboard. This rule fires on any oversized ICMP echo request (packet size over 1000 bytes).

```
sudo nano /etc/suricata/rules/local.rules
alert icmp any any -> any any (msg:"Ping of Death - Oversized ICMP"; dsize:>1000; itype:8;
sid:1000002; rev:1;)
```

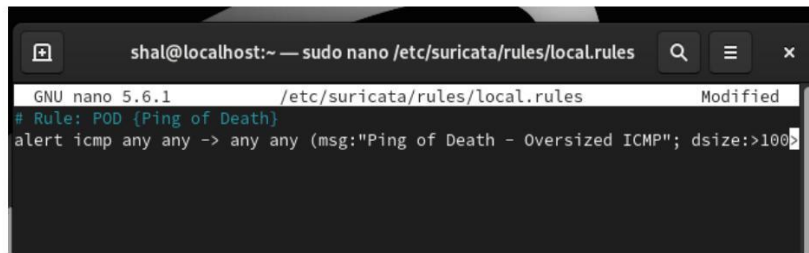
```
# Create local rules file
sudo nano /etc/suricata/rules/local.rules

# Rule: POD {Ping of Death}
alert icmp any any -> any any (msg:"Ping of Death - Oversized ICMP";
dsize:>1000; itype:8; sid:1000002; rev:1;)

# Restart Suricata
sudo systemctl daemon-reload
sudo systemctl restart suricata

# Testing from Kali
sudo ping -M want -s 65500 <TARGET_IP>
-----

status
```



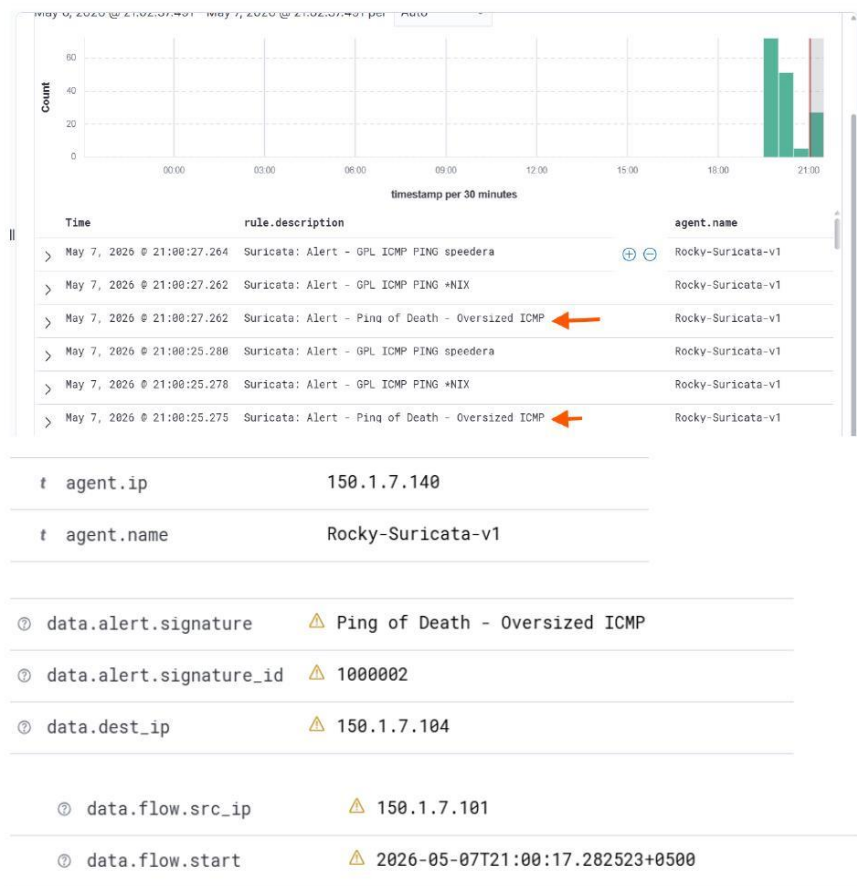
local.rules file open in nano showing the Ping of Death signature

Testing the Ping of Death signature

From Kali Linux, send an oversized ICMP packet to trigger the rule:

```
sudo ping -M want -s 65500 150.1.7.104
```

```
(kali@kali)-[~]
$ sudo ping -M want -s 65500 150.1.7.104
[sudo] password for kali:
PING 150.1.7.104 (150.1.7.104) 65500(65528) bytes of data.
65508 bytes from 150.1.7.104: icmp_seq=1 ttl=64 time=3.59 m
65508 bytes from 150.1.7.104: icmp_seq=2 ttl=64 time=4.48 m
65508 bytes from 150.1.7.104: icmp_seq=3 ttl=64 time=4.69 m
65508 bytes from 150.1.7.104: icmp_seq=4 ttl=64 time=4.52 m
65508 bytes from 150.1.7.104: icmp_seq=5 ttl=64 time=2.49 m
65508 bytes from 150.1.7.104: icmp_seq=7 ttl=64 time=10.8 m
65508 bytes from 150.1.7.104: icmp_seq=8 ttl=64 time=3.35 m
65508 bytes from 150.1.7.104: icmp_seq=9 ttl=64 time=2.92 m
^C
— 150.1.7.104 ping statistics —
9 packets transmitted, 8 received, 11.111% packet loss, ti
rtt min/avg/max/mdev = 2.492/4.599/10.763/2.445 ms
(kali@kali)-[~]
$
```



Kali sending oversized ping (top), Wazuh dashboard showing Ping of Death alerts with data.dest_ip and data.flow.src_ip fields confirmed (bottom)

This screenshot was key for debugging the IP path issue. It confirmed that the correct JSON fields in the Wazuh alert are data.dest_ip for the victim and data.flow.src_ip for the attacker, not data.src_ip as sometimes assumed.

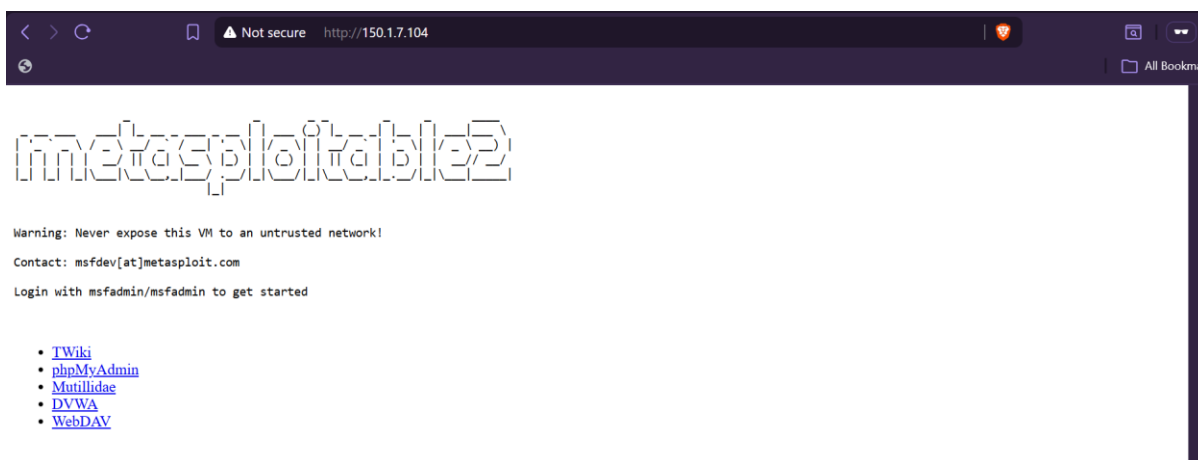
SlowLoris Signature (SID 1000011)

SlowLoris works by opening many TCP connections to an HTTP server and sending partial headers slowly, keeping them alive indefinitely. The attack exhausts the server's connection pool without ever completing a request. The detection strategy is to alert when a single source IP establishes more than 20 SYN connections to port 80 within 10 seconds.

```
alert tcp any any -> $HOME_NET 80 (msg:"SLOWLORIS High Connection Rate to HTTP Port"; \
flow:to_server; \ flags:S; \ threshold:type threshold, track by_src, count 20, seconds
10; \ sid:1000011; rev:2;)
```

The flags:S keyword matches only SYN packets, and the threshold option tracks by source IP so the rule only fires when one source hits the count limit, not from aggregate traffic across all clients.

```
sudo systemctl daemon-reload
sudo systemctl restart suricata
```

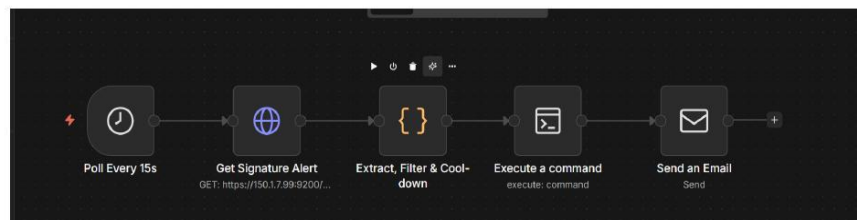
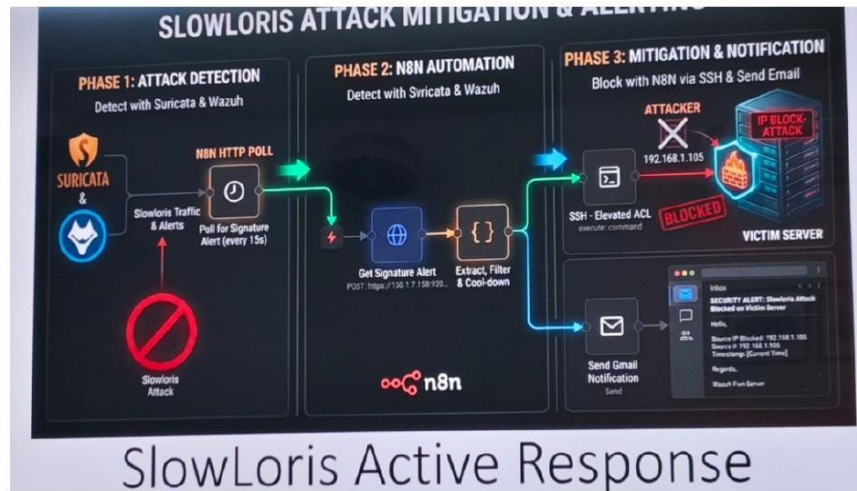


>	May 7, 2026 @ 20:52:49.069 Suricata: Alert - GPL ICMP PING *NIX	Rocky-Suricata-v1
>	May 7, 2026 @ 20:52:47.081 Suricata: Alert - GPL ICMP PING *NIX	Rocky-Suricata-v1
>	May 7, 2026 @ 20:52:47.079 Suricata: Alert - GPL ICMP PING *NIX	Rocky-Suricata-v1
>	May 7, 2026 @ 20:52:46.481 Suricata: Alert - GPL ICMP PING *NIX	Rocky-Suricata-v1

Metasploitable web interface at 150.1.7.104 (top), Wazuh security events showing SlowLoris alerts firing in real time (bottom)

The n8n SOAR Workflow

With Suricata detecting attacks and Wazuh indexing the alerts into Elasticsearch, the final piece is automating the response. The n8n workflow has five nodes that run in sequence every 15 seconds.



Automtion for slowloris attack and a SOAR workflow, that blocks the tagetting IP and give us an email

Three-phase pipeline diagram (top): detection via Suricata/Wazuh, automation via n8n, mitigation via SSH iptables and email. n8n workflow editor showing all five nodes (bottom).

Node 1: Poll Every 15s

A Schedule Trigger node fires every 15 seconds. This is the heartbeat of the workflow. Most executions will stop at the code node with no output because there is no active attack. That is expected and normal.

Node 2: Get Signature Alert

An HTTP POST request queries the Wazuh Elasticsearch API for the most recent alert matching signature ID 1000011, filtered to only the last 30 seconds. This time window filter is critical: without it, old alerts from previous attacks would be refetched indefinitely.

```
POST https://150.1.7.99:9200/wazuh-alerts-4.x-*/_search {  "size": 1,  "sort": [{
"@timestamp": { "order": "desc" } }],  "query": {    "bool": {      "must": [        {
"match": { "data.alert.signature_id": "1000011" } }      ],      "filter": [        {
"range": { "@timestamp": { "gte": "now-30s" } } }      ]    }  } }
```

Node 3: Extract, Filter and Cool-down

A JavaScript code node applies three gates before allowing execution to continue. Any one of them returns an empty array, which stops the entire workflow chain cleanly:

- No hits: the Wazuh query returned nothing, so there is no active attack.
- Alert too old: the alert timestamp is more than 20 seconds ago, meaning the attack likely already ended.
- Already processed: the alert's unique Elasticsearch document ID matches the last processed ID stored in workflow static data. This prevents the same attack event from triggering duplicate blocks and emails.

```
const hits = items[0].json.hits.hits; if (hits.length === 0) return []; const alertTime =
new Date(hits[0]._source['@timestamp']); const timeDiffSeconds = (new Date() - alertTime) /
1000; if (timeDiffSeconds > 20) return []; const alertId = hits[0]._id; const staticData =
$getWorkflowStaticData('global'); if (staticData.lastProcessedAlertId === alertId) return [];
staticData.lastProcessedAlertId = alertId; return [{  json: {    src_ip:
hits[0]._source.data.flow.src_ip,    dest_ip: hits[0]._source.data.dest_ip,    alert_age:
timeDiffSeconds  } }];
```

Always Output Data must be OFF on this node. If it is ON, n8n will pass an empty item downstream even when the code returns [], causing SSH and email to fire with blank fields on every poll.

Node 4: Execute a Command via SSH

An SSH node connects to Metasploitable (150.1.7.104) and runs an iptables command that first checks whether the attacker's IP is already blocked, and only adds the DROP rule if it isn't. This prevents duplicate rules from accumulating over multiple alerts for the same attack.

```
echo 'msfadmin' | sudo -S iptables -L INPUT -n | grep -q "{{ $('Extract, Filter & Cool-
down').item.json.src_ip }}" || echo 'msfadmin' | sudo -S iptables -A INPUT -s {{ $('Extract,
Filter & Cool-down').item.json.src_ip }} -j DROP
```

Node 5: Send an Email

An SMTP email node sends a structured HTML alert with the attacker IP, victim IP, and alert age. All values are pulled from the code node output using explicit node references to avoid resolving to undefined.

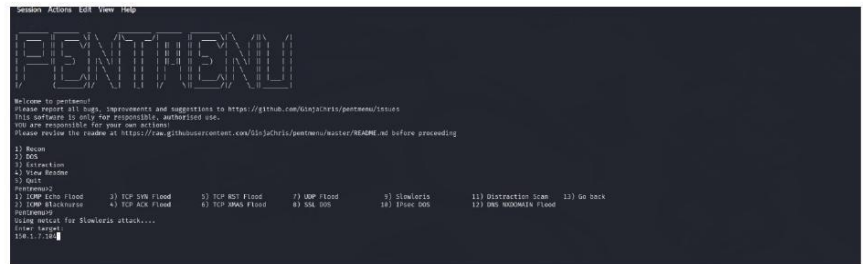
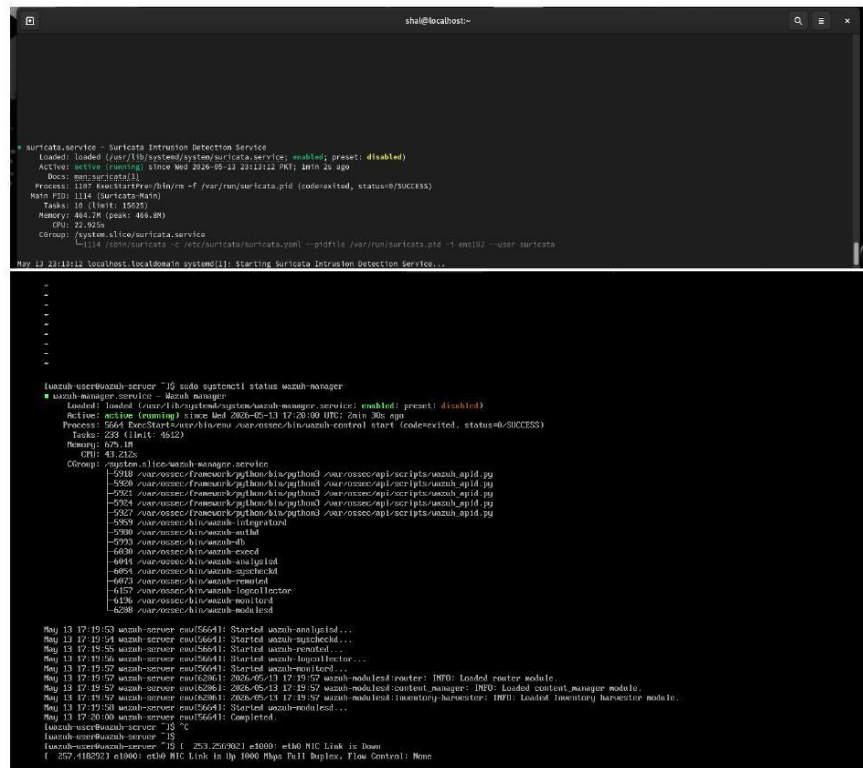
Launching the Attack and Verifying the Response

Setting up pentmenu on Kali

pentmenu is a bash-based tool that makes it easy to launch various network attacks including SlowLoris. Install it on Kali:

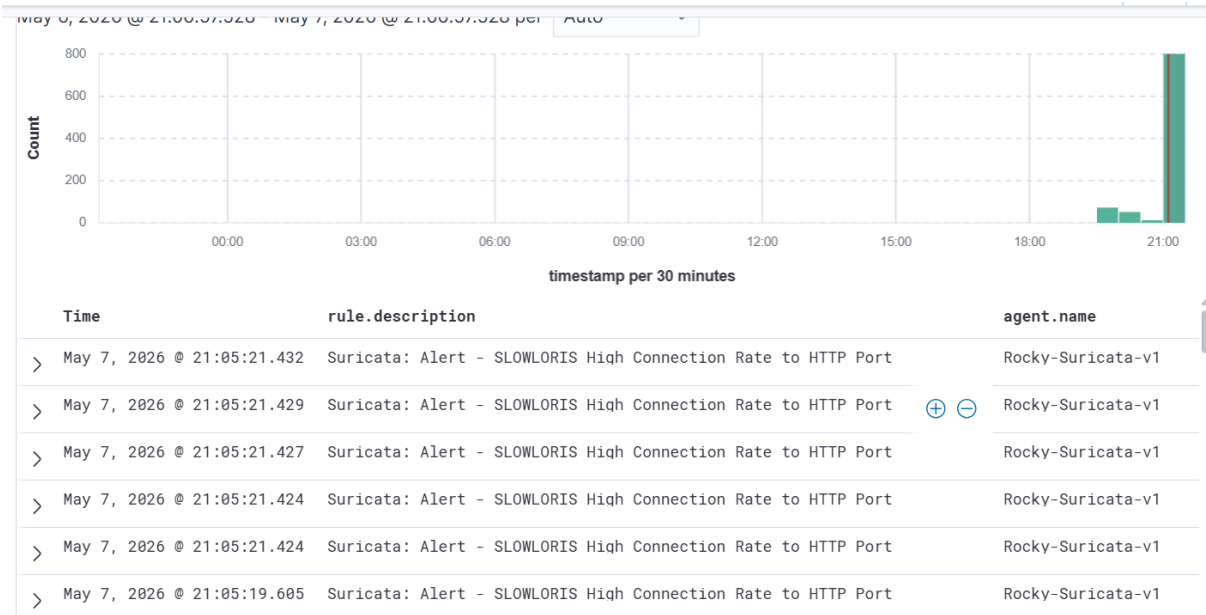
```
git clone https://github.com/GinjaChris/pentmenu.git
cd pentmenu && ./pentmenu
```

Suricata and Wazuh manager both confirmed running (top), pentmenu launched on Kali with SlowLoris attack targeting 150.1.7.104 (bottom)



SlowLoris alerts appearing in Wazuh

```
kali@kali: ~/pentmenu
Session Actions Edit View Help
Slowloris attack ongoing ... this is connection 1975, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1976, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1977, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1978, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1979, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1980, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1981, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1982, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1983, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1984, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1985, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1986, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1987, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1988, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1989, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1990, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1991, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1992, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1993, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1994, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1995, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1996, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1997, interval is 6 seconds
Slowloris attack ongoing ... this is connection 1998, interval is 6 seconds
```



SlowLoris attack in progress on Kali (top terminal), Wazuh security events dashboard showing multiple SLOWLORIS High Connection Rate alerts from Rocky-Suricata-v1 (bottom)

n8n executes and blocks the IP

The top screenshot shows a terminal window titled "n8n@kali: ~" with a "PENTEST" banner. It displays a Slowloris attack script running on a Kali Linux machine. The script is configured to attack a target IP (192.168.1.100) using a range of ports (80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99). The output shows the attack is ongoing, with connections established and data being sent.

The bottom screenshot shows the n8n workflow editor. The workflow is titled "Suricata automated Response" and is published. It consists of the following steps:

- Poll Every 15s**: A trigger node that polls every 15 seconds.
- Get Signature Alert**: A node that receives alerts from Suricata. The alert details shown are: Post: https://n8n.io/200...
- Extract, Filter & Cool-down**: A node that processes the alert data.
- Execute a command**: A node that executes a command based on the alert data.
- Send an Email**: A node that sends an email notification.

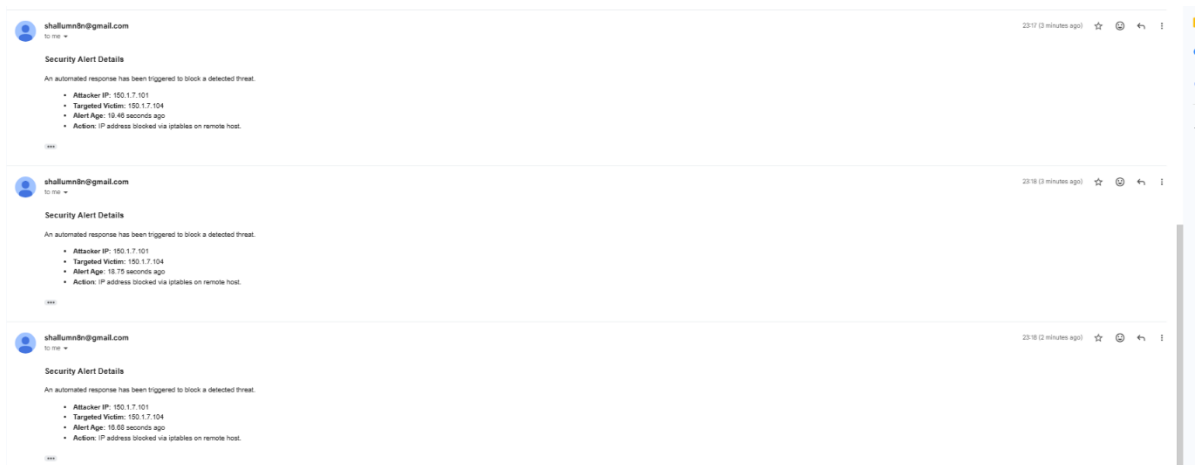
The workflow is shown as successful, with a status of "Succeeded in 14s | ID#273 | Version 7b5c19c4". The left sidebar shows a list of recent executions, all of which were successful.

SlowLoris still running with 2000+ connections (top), n8n execution history showing a successful 14-second run with all 5 nodes completing (bottom)

The execution that took 14 seconds was the real one. Every other execution in the history took under 500ms because those stopped at the code node with no data. This is the expected pattern.

Verification: iptables rule applied and email received

```
target      prot opt source      destination
root@metasploitable:/home/msfadmin# iptables -L INPUT -n --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source      destination
1 DROP          all  --  150.1.7.101  0.0.0.0/0
root@metasploitable:/home/msfadmin# _
```



Metasploitable showing iptables DROP rule for 150.1.7.101 (Kali) added successfully (top), Gmail inbox showing three structured security alert emails with attacker IP, victim IP, and alert age populated correctly (bottom)